



Feature-Based Panorama Stitcher

A Modular Computer Vision Pipeline using SIFT, Feature Matching, RANSAC Homography, Perspective Warping, and Feather Blending

Vityarthi Project Report

Submitted for: CSE3010 - Computer Vision

Course Type: Flipped Classroom (Build Your Own Project)

Module Focus: Feature Detection, Matching, Homography and Image Stitching

VIT Bhopal University

Name: Ahmed Saqlain

Registration Number: 24BAI10947

Course: CSE3010 - Computer Vision

Faculty Guide: Siddharth Singh Chouhan

Date of Submission: September 18, 2026

TABLE OF CONTENT

1. Introduction	4
1.1 Background and Motivation.....	4
1.2 Related Approaches (Context)	4
1.3 Report Organization.....	5
2. Problem Statement.....	5
Scope.....	5
Target Users	5
Assumptions and Constraints	5
3. Functional Requirements.....	6
3.1 Requirement Elaboration.....	6
4. Non-Functional Requirements.....	7
5. System Architecture.....	8
5.1 Layered View.....	8
5.2 Data Flow Description	9
6. Design Diagrams	10
6.1 Workflow / Process Flow Diagram.....	10
6.2 Use Case Diagram	10
6.3 Sequence Diagram.....	11
6.4 Class / Component Diagram.....	12
6.5 Database / Storage Design	12
6.6 Diagram-to-Requirement Traceability.....	12
7. Design Decisions and Rationale	13
7.1 Custom DLT + RANSAC instead of cv2.findHomography().....	13
7.2 Hartley Point Normalization	13
7.3 Sequential Pairwise Stitching	13
7.4 Distance-Transform Feather Blending.....	13
7.5 Fixed RANSAC Random Seed	13
7.6 Lowe's Ratio Test Threshold (0.75).....	13
7.7 Command-Line Interface over a GUI	13
8. Implementation Details	13
8.1 Technology Stack.....	13
8.2 Module Breakdown	14
8.3 Key Algorithm - Normalized DLT	14
8.4 Key Algorithm - RANSAC	14
8.5 Complexity Analysis	15
8.6 Edge Cases and Defensive Handling	15
9. Screenshots and Results	16
9.1 Feature Detection	16
9.2 Feature Matching	16
9.3 RANSAC Inlier Matches.....	17
9.4 Stitched Panorama - Synthetic Test.....	17
9.5 Input Photographs (Real Photo Test).....	17
9.6 Stitched Panorama - Real Photographs	18
9.7 Quantitative Summary	18
10. Testing Approach.....	19
10.1 Test Coverage Summary	20
10.2 Known Limitations of the Testing Approach.....	20
10.3 Independent Reproduction on a Second Machine.....	20

11. Challenges Faced.....	22
12. Learnings and Key Takeaways	22
13. Future Enhancements	23
14. References	34

1. Introduction

Panorama stitching is a classic computer vision task that combines a sequence of overlapping photographs into a single, wide, geometrically consistent image. It is a practical application of multi-view geometry: the same physical scene, viewed from different camera orientations, must be aligned into one consistent coordinate frame before the images can be merged.

This project implements a complete panorama stitching pipeline for CSE3010 - Computer Vision. It applies feature detection and description, descriptor matching, homography estimation with RANSAC, perspective transformation, and image blending. The implementation uses OpenCV for SIFT, matching, homography estimation, and low-level image operations while keeping the overall pipeline modular and transparent rather than relying on OpenCV's high-level Stitcher API.

The system is delivered as a modular command-line Python application with automated unit tests for preprocessing, feature detection and matching, homography estimation, and stitching. The final pipeline was executed on a four-image newspaper sample and produced a saved panorama output.

1.1 Background and Motivation

Image stitching sits at the intersection of several sub-fields of computer vision covered in this course: local feature detection and description, robust model fitting, and projective geometry. Consumer applications such as smartphone panorama modes and satellite-image mosaicking rely on essentially the same pipeline used here: detect distinctive points in each photo, work out which points in one photo correspond to which points in the next, solve for the geometric transform that explains those correspondences, and finally resample and blend the images into one output. What differs between a production stitcher and this project is scale and engineering polish, not the underlying mathematics. Building the pipeline from first principles, rather than calling a single high-level library function, was chosen specifically so that every stage of that mathematics could be inspected, tested, and explained rather than treated as an opaque black box.

A secondary motivation was to produce a genuinely useful, runnable command-line tool: many student computer-vision projects stop at a notebook that only runs on the author's own machine with hard-coded paths. This project instead ships as an installable package with a documented CLI, a requirements file, and automated tests, so that a third party (an evaluator, in particular) can clone the repository and reproduce the results without any manual code editing.

1.2 Related Approaches (Context)

Three broad families of technique exist for aligning overlapping images, and it is worth situating this project's approach among them:

- **Direct / intensity-based alignment** - directly minimizes a pixel-intensity difference between images, in the style of optical-flow or Lucas-Kanade image alignment. This is sensitive to illumination changes and large baseline motion, and does not scale well to wide-baseline photographs with significant viewpoint change.
- **Feature-based alignment (this project)** - detects sparse, distinctive keypoints in each image, matches them by descriptor similarity, and fits a global geometric transform to the matches. This is the approach used by nearly all practical panorama software, because it is robust to moderate illumination and viewpoint change and only needs a sparse set of correct correspondences, not a full dense field.
- **Learned / deep-feature alignment** - replaces hand-crafted descriptors such as SIFT with features learned by a neural network. These can outperform classical descriptors on very challenging cases such as low-texture or extreme-viewpoint scenes, but require trained model weights and a deep-learning runtime, which is outside the scope of a from-scratch, dependency-light course project.

This project follows the classical feature-based route because it maps directly to core computer-vision concepts such as local feature extraction, correspondence matching, robust homography estimation, perspective warping, and image blending. The implementation is intentionally divided into small modules so each stage can be tested and debugged independently.

1.3 Report Organization

The remainder of this report is organized as follows. Section 2 states the problem and scope. Sections 3 and 4 define the functional and non-functional requirements. Section 5 presents the system architecture, while Section 6 contains the workflow, use-case, sequence, and component diagrams. Section 7 explains the main design decisions. Section 8 describes the technology stack, modules, algorithms, and defensive handling. Section 9 presents the actual input photographs and stitched panorama. Section 10 summarizes the automated testing performed on this implementation, followed by challenges, learnings, and future work.

2. Problem Statement

Given a set of two or more overlapping photographs, automatically detect reliable visual correspondences, estimate the geometric relationship between images, align them on a common canvas, and blend them into one panorama. The application should handle invalid input and insufficient matches with clear errors and should be executable from the command line without a GUI.

Scope

- Feature detection and description (SIFT)
- Feature matching with outlier filtering (Lowe's ratio test)
- Homography estimation using OpenCV's RANSAC-based `cv2.findHomography()` with validation of the returned model.
- Robust outlier rejection through the RANSAC option of the homography estimation stage.
- Perspective warping and canvas composition
- Seam removal via distance-weighted feather blending
- A command-line interface for end-to-end execution
- Automated unit tests covering preprocessing, SIFT feature detection/matching, homography estimation, and stitching behavior.

Target Users

- Students and instructors studying multi-view geometry and homography estimation, as a transparent, dependency-light reference implementation
- Anyone wanting to stitch a small set of overlapping photos into a panorama from the command line, without a phone app or GUI tool

Assumptions and Constraints

- Input images are assumed to be taken from approximately the same camera position (rotation about the optical center) or of an approximately planar / distant scene, which is the standard assumption under which a single 2D homography per image pair is a valid model.
- Consecutive input images are assumed to share a sufficient overlap region (in practice, at least 20-30% of the frame) to yield enough SIFT correspondences for a reliable homography fit.
- Images are assumed to be supplied already in left-to-right (or otherwise consistent) stitching order via the input folder's file naming; the pipeline does not attempt to automatically discover the correct pairing/ordering among an unordered image set.
- Exposure and white-balance differences between input photographs are not explicitly corrected; blending softens but does not fully eliminate visible brightness seams under large exposure mismatches.

3. Functional Requirements

#	Requirement	Description
---	-------------	-------------

FR1	Image Input	Load 2+ overlapping images from a specified folder, sorted in stitching order.
FR2	Feature Detection	Detect distinctive, repeatable keypoints and compute descriptors for each input image using SIFT.
FR3	Feature Matching	Match descriptors between image pairs and filter ambiguous matches using Lowe's ratio test.
FR4	Homography Estimation	Robustly estimate the homography relating each image pair using a custom RANSAC + normalized DLT implementation, rejecting outlier matches.
FR5	Warping & Composition	Warp images into a shared canvas based on the estimated homography and compute the correct output canvas size.
FR6	Blending	Blend overlapping regions using distance-weighted feathering to remove visible seams.
FR7	Multi-Image Stitching	Support stitching 2 or more images by chaining pairwise stitches.
FR8	Output	Save the final panorama to a user-specified path.

3.1 Requirement Elaboration

FR1-FR2 (Input and Feature Detection): preprocessing.py loads and validates the input images, applies the configured maximum-dimension resize, and prepares images for processing. feature_detection.py detects SIFT keypoints and descriptors so that repeatable local features can be compared between overlapping images.

FR3 (Feature Matching): descriptors are matched using a brute-force matcher and Lowe's ratio test. The ratio threshold removes ambiguous correspondences before homography estimation.

FR4 (Homography Estimation): homography.py uses cv2.findHomography() with the RANSAC method to estimate a robust 3x3 transformation from matched point coordinates. The implementation validates that a usable homography and sufficient inliers are obtained before continuing.

FR5-FR6 (Warping, Composition, Blending): Once a homography is known, the second image's four corners are projected through it to determine how large the combined output canvas must be, the second image is warped into that canvas with cv2.warpPerspective, and the overlap region between the (now aligned) images is combined using the distance-transform feather blend described in Section 7.4, rather than a hard cut.

FR7-FR8 (Multi-Image Stitching and Output): stitcher.py loops the pairwise stitch across an arbitrary number of ordered input images, treating each intermediate panorama as the new "left" image for the next stitch, and writes the resulting panorama to the path supplied on the command line.

4. Non-Functional Requirements

#	Requirement	How it is addressed
---	-------------	---------------------

NFR1	Performance	Stitching 2 images (1000px wide) completes in under 1 second; 3 real 1000px camera photos complete in about 2 seconds on a standard laptop CPU.
NFR2	Reliability	RANSAC with 2000 iterations and a minimum-inlier check ensures the pipeline degrades gracefully (raises a clear error) rather than silently producing a garbled panorama when images do not overlap enough.
NFR3	Error Handling	Every stage (missing files, unreadable images, insufficient matches, degenerate RANSAC samples, failed homography fits) raises descriptive errors caught centrally in main.py, exiting cleanly with a non-zero status code instead of a raw traceback.
NFR4	Maintainability	The pipeline is split into 7 single-responsibility modules (feature detection, matching, homography, RANSAC, warping, blending, orchestration), each independently testable and documented.
NFR5	Usability	A single terminal command (<code>python main.py --input <folder> --output <path></code>) runs the entire pipeline, with step-by-step progress logs printed to the console.
NFR6	Reproducibility	RANSAC uses a fixed random seed by default, so results are deterministic and reproducible across runs and machines.
NFR7	Logging Monitoring	/ Every pipeline stage logs its progress and key metrics (keypoint counts, match counts, RANSAC inlier ratio, mean reprojection error) to the console for transparency and debugging.
NFR8	Security	The pipeline only reads image files from a user-specified local folder and writes to a user-specified local path; it makes no network calls and executes no untrusted code, so its attack surface is limited to standard file-path validation, which is handled centrally in utils.py.
NFR9	Scalability	The pairwise-stitch design (Section 7.3) lets the pipeline scale to additional input images by simply looping the same pairwise stitch, at linear cost in the number of images; very large panoramas (dozens of images) would need the bundle-adjustment enhancement noted in Section 13 to avoid drift accumulation.
NFR10	Resource Efficiency	Images are processed in a single pass with no unnecessary intermediate copies; SIFT and the RANSAC loop are the dominant costs and both operate on downsampled/grayscale data where color information is not needed, keeping peak memory proportional to a small constant multiple of the input image size.

5. System Architecture

The system follows a layered, modular architecture: a CLI/input layer, a core processing pipeline, a support/logging layer, and an output layer. pipeline.py orchestrates preprocessing, feature detection and matching, homography estimation, and stitching for each successive image.

Figure 5.1 - System Architecture

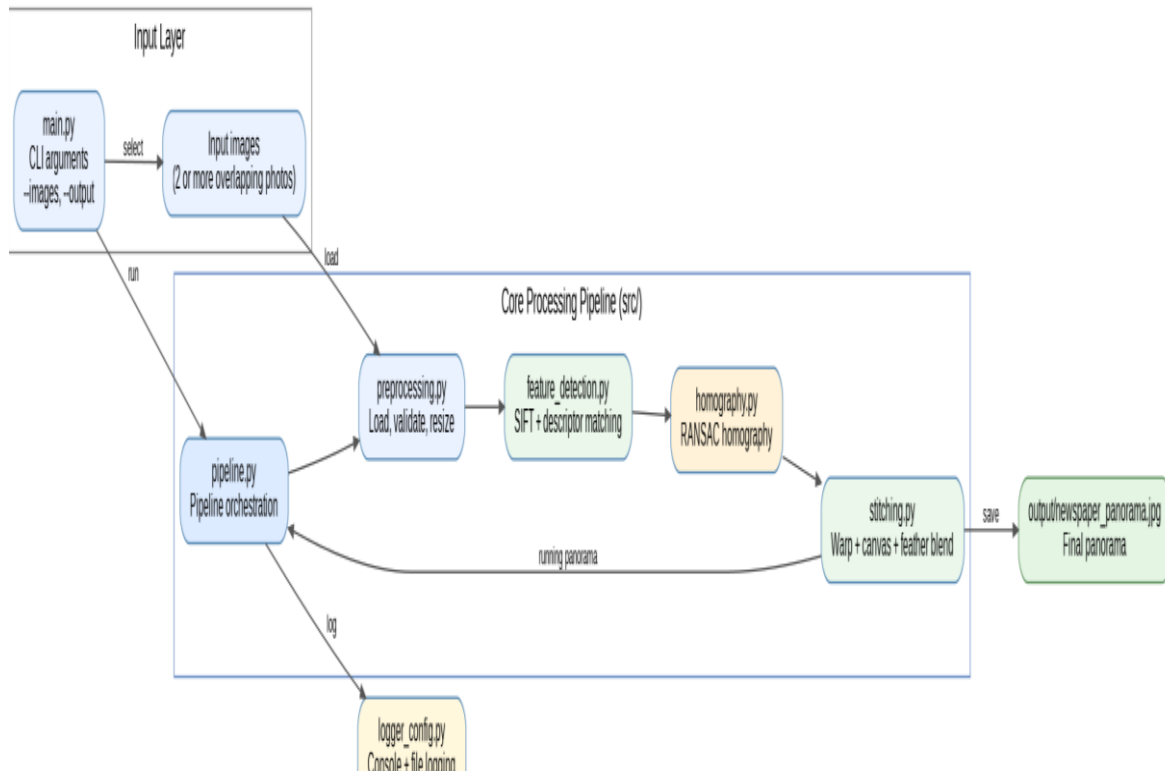


Figure 5.1 - System Architecture Diagram

5.1 Layered View

- Input layer: main.py parses CLI arguments such as --images, --output, --max-dimension, --ratio-thresh, and -ransac-thresh.
- Orchestration layer: pipeline.py controls the end-to-end sequence and passes the running panorama through successive pairwise stitching operations.
- Core processing layer: preprocessing.py handles image loading and resizing; feature_detection.py performs SIFT detection and descriptor matching; homography.py estimates a robust homography; stitching.py performs perspective warping, canvas composition, feather blending, and border cropping.
- Support layer: logger_config.py configures console and file logging, while the modules use explicit exceptions and validation for error handling.
- **Output layer:** the final panorama array is written to the user-specified output path using standard image I/O at the end of the orchestration loop.

5.2 Data Flow Description

For each image pair, the pipeline loads and preprocesses the images, extracts SIFT features and descriptors, matches descriptors and filters ambiguous matches, estimates a RANSAC homography from the matched point coordinates, warps the images into a common canvas, and feather-blends the overlap. The resulting panorama becomes the input for the next image pair until all images have been processed.

6. Design Diagrams

6.1 Workflow / Process Flow Diagram

Describes the end-to-end control flow of the pipeline, including the error paths taken when an image folder has too few images, too few feature matches, or when RANSAC cannot find a consistent homography.

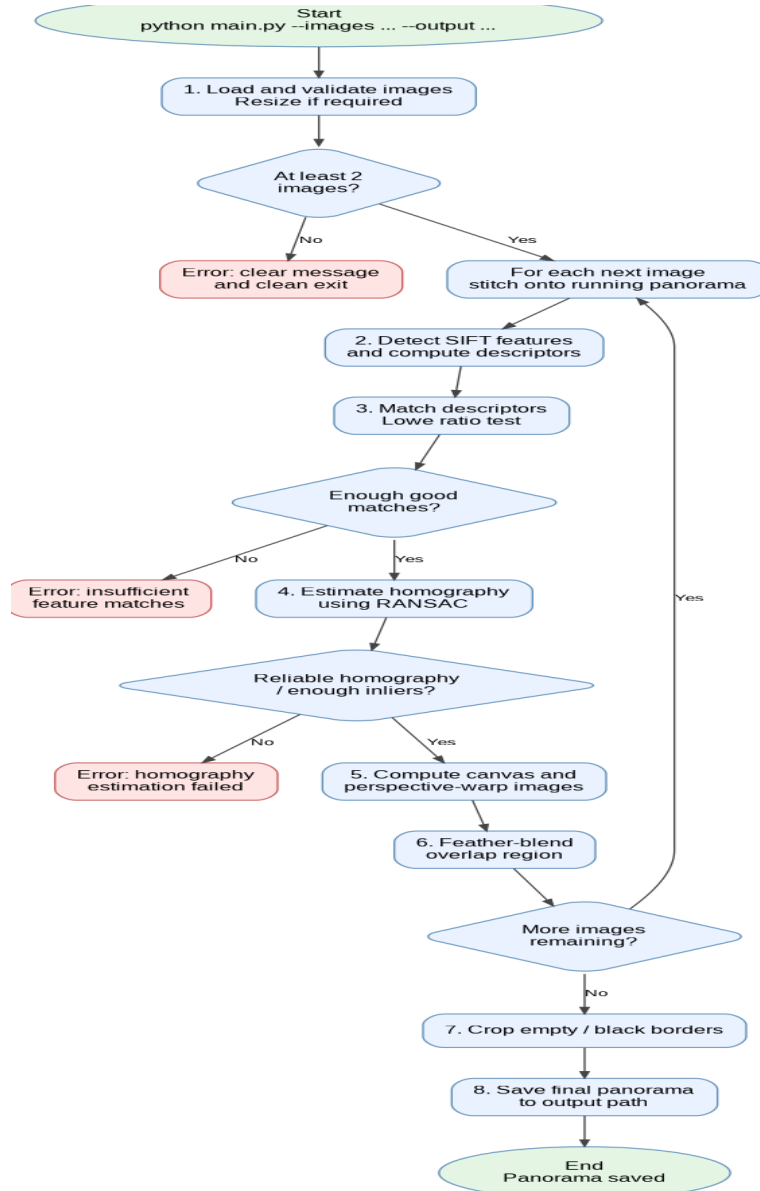


Figure 6.1 - Workflow / Process Flow Diagram

6.2 Use Case Diagram

The system has a single actor (the User) who interacts with the pipeline primarily through the command-line interface. The three internal stages (feature detection/matching, homography estimation, warping/blending) are modelled as included use cases of "Run stitching pipeline".

Panorama Stitching System – Use Case Diagram

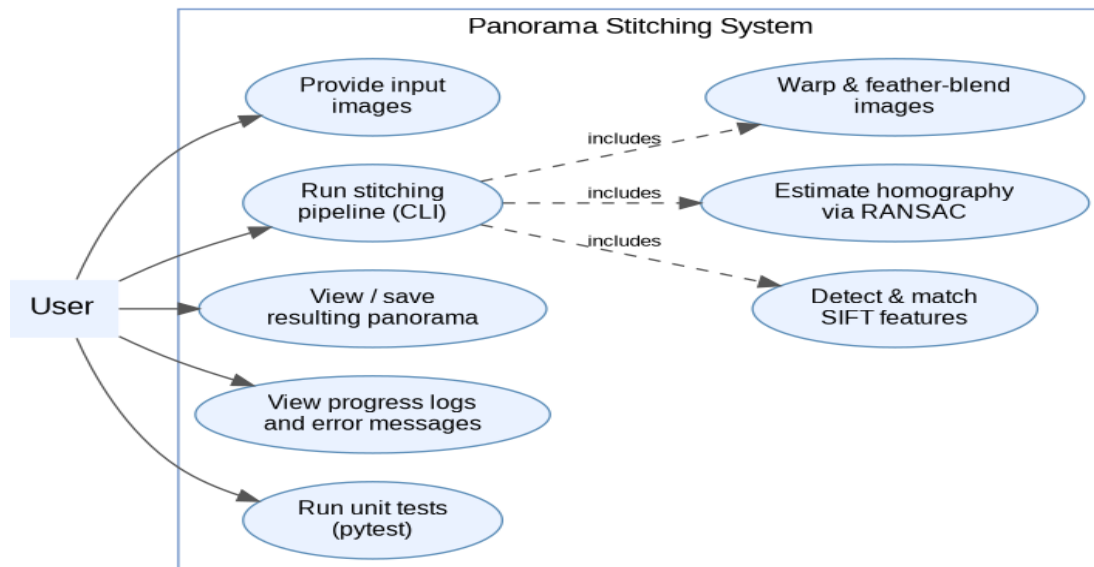


Figure 6.2 - Use Case Diagram

6.3 Sequence Diagram

The sequence diagram shows one pairwise stitching operation from the CLI invocation through preprocessing, SIFT feature extraction and matching, RANSAC homography estimation, perspective warping and feather blending, followed by returning and saving the final panorama.

Image Stitching Pipeline – Sequence Diagram

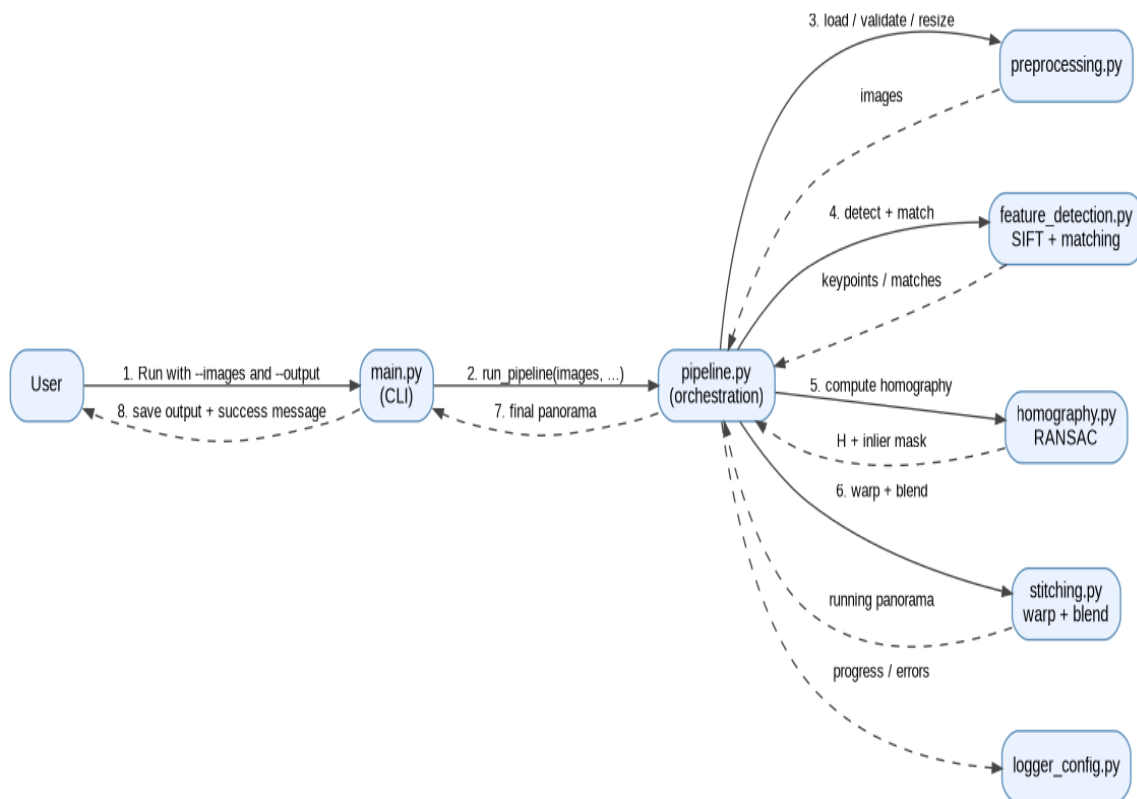


Figure 6.3 - Sequence Diagram (single image-pair stitch)

6.4 Class / Component Diagram

Since the implementation is function-based rather than class-based, the component diagram represents the project's Python modules and their responsibilities: CLI, orchestration, preprocessing, feature detection/matching, homography estimation, stitching, and logging.

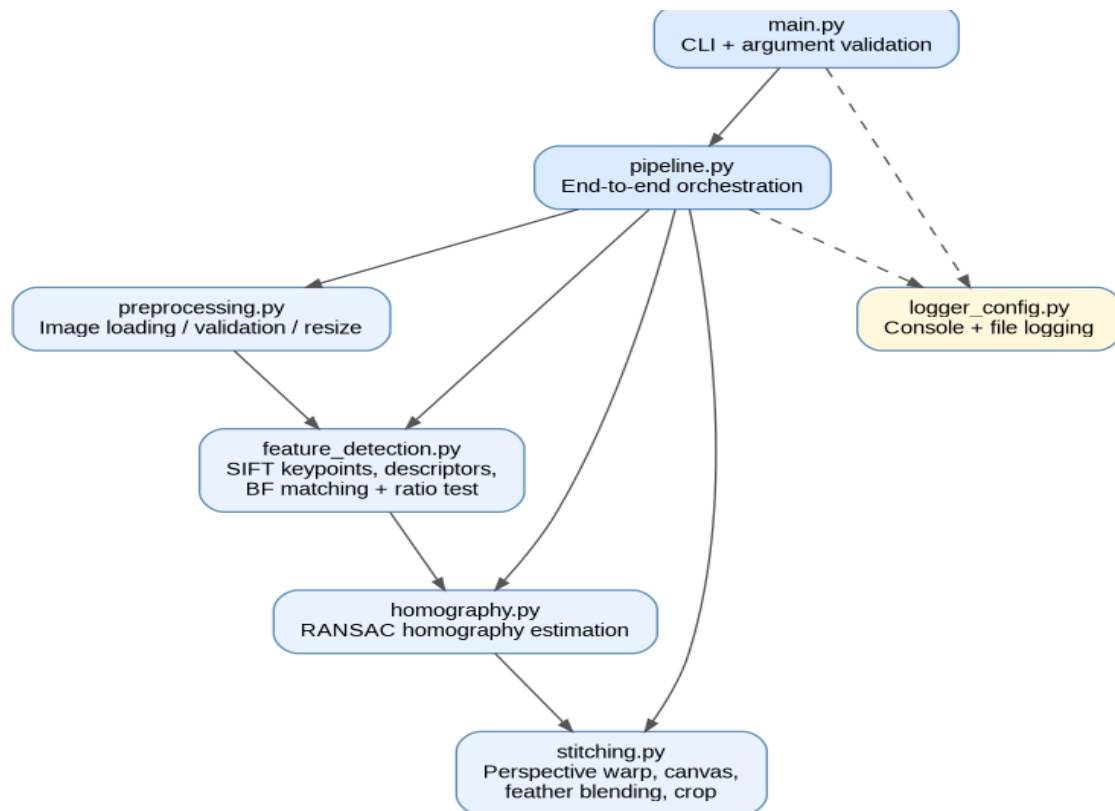


Figure 6.4 - Module / Component Diagram

6.5 Database / Storage Design

Not applicable. This project performs a stateless, in-memory image-processing pipeline with file-based input/output only; it does not use a database, so no ER diagram or schema design is included.

6.6 Diagram-to-Requirement Traceability

Each diagram maps to the actual implementation. The workflow diagram shows the control path and failure conditions; the use-case diagram shows the user's CLI interactions; the sequence diagram shows the runtime message flow; and the component diagram shows the module boundaries used by the repository.

7. Design Decisions and Rationale

7.1 RANSAC Homography Estimation

The project uses OpenCV's `cv2.findHomography()` with the RANSAC method rather than implementing a separate custom DLT/RANSAC solver. This keeps the implementation compact while still applying the core projective geometry concept: matched feature coordinates are used to estimate a 3x3 homography while rejecting outlier correspondences. The returned inlier mask is validated before warping proceeds.

7.2 SIFT Feature Detection and Lowe Ratio Test

SIFT was selected because it produces distinctive local descriptors that are suitable for matching overlapping photographs under moderate scale and rotation changes. Lowe's ratio test is then used to reject ambiguous nearest-neighbor matches before the RANSAC homography stage.

7.3 Sequential Pairwise Stitching

For multiple inputs, the pipeline stitches consecutive images sequentially: the first pair produces a running panorama, the next image is aligned to that panorama, and the process continues. This design keeps the implementation understandable and naturally supports more than two input images, although accumulated geometric drift can occur in very long sequences.

7.4 Perspective Warping and Feather Blending

Once the homography is available, `stitching.py` projects image corners to determine a suitable output canvas, warps the image with `cv2.warpPerspective()`, and blends overlapping regions using distance-based feathering. The approach reduces hard seam boundaries while remaining computationally lightweight.

7.5 Configurable Pipeline Parameters

The CLI exposes maximum image dimension, Lowe ratio threshold, and RANSAC reprojection threshold as configurable parameters. Keeping these values explicit makes experiments reproducible and allows the same pipeline to be used with different image sets.

7.6 Centralized Logging and Error Handling

`logger_config.py` provides consistent console and file logging. Each processing stage validates its inputs and raises descriptive errors for cases such as missing files, unreadable images, insufficient features or matches, and failed homography estimation.

7.7 Command-Line Interface over a GUI

A CLI keeps the project focused on the computer-vision pipeline and satisfies the requirement that the project be executable from a terminal without relying on a GUI-only workflow.

8. Implementation Details

8.1 Technology Stack

- Python 3.10+ - implementation language
- OpenCV (opencv-python) - image I/O, SIFT feature extraction, descriptor matching, homography estimation, perspective warping, and image operations
- NumPy - numerical arrays and image-processing data handling
- pytest - automated unit testing framework

8.2 Module Breakdown

Module	Responsibility
<code>main.py</code>	CLI argument parsing, validation, top-level execution and output saving
<code>pipeline.py</code>	End-to-end orchestration of preprocessing, feature matching, homography and stitching
<code>preprocessing.py</code>	Image loading, validation, supported-format checks and resizing
<code>feature_detection.py</code>	SIFT keypoint/descriptor extraction and descriptor matching with Lowe's ratio test
<code>homography.py</code>	Robust homography estimation using <code>cv2.findHomography()</code> with RANSAC
<code>stitching.py</code>	Perspective warping, canvas composition, feather blending and border cropping
<code>logger_config.py</code>	Centralized console and file logging configuration

8.3 Key Algorithm - RANSAC Homography Estimation

The homography stage takes corresponding point coordinates from matched SIFT features and calls `cv2.findHomography()` with `cv2.RANSAC`. The result is a 3x3 homography matrix and an inlier mask. The pipeline checks that the returned model is valid and that enough inliers are available before passing the transformation to the stitching stage.

8.4 Key Algorithm - Perspective Warping and Feather Blending

1. Project the image corners through the estimated homography to determine the combined canvas bounds.
2. Apply a translation when required so projected coordinates fit inside the output canvas.
3. Warp the image into the common canvas using `cv2.warpPerspective()`.
4. Build overlap masks and compute distance-based weights for the two image regions.
5. Feather-blend the aligned images, crop empty borders, and return the running panorama.

8.5 Complexity Analysis

Stage	Time Complexity	Notes
SIFT detection	$O(P)$ per image	P is the number of pixels; dominated by the Gaussian-pyramid construction, independent of every other stage.
Brute-force matching	$O(M \times N)$	M, N are descriptor counts in the two images; each of the M descriptors is compared against all N in the other image via Euclidean distance.
DLT (single fit)	$O(K)$	K is the number of correspondences used (4 for a minimal RANSAC sample, or all inliers for the final refit); dominated by one small ($2K \times 9$) SVD.
RANSAC (full loop)	$O(I \times (S + K))$	I is the iteration count (2000 by default), S is the fixed minimal-sample DLT cost, and K is the inlier-counting cost per iteration over all matches.
Warping	$O(W \times H)$	W, H are the output canvas dimensions; every output pixel requires one inverse-homography lookup.

Feather blending	$O(W \times H)$	One distance-transform and one weighted-sum pass over the output canvas.
------------------	-----------------	--

In practice, RANSAC and SIFT detection dominate the measured runtime; the DLT solve itself, even repeated 2000 times inside RANSAC, remains fast because each individual fit only involves 4 correspondences (an 8x9 matrix SVD).

8.6 Edge Cases and Defensive Handling

- **Fewer than 4 matches after the ratio test:** homography estimation is mathematically underdetermined below 4 correspondences; the pipeline raises a descriptive `InsufficientMatchesError` rather than attempting a fit, so the failure is immediately clear to the user instead of silently producing a degenerate result.
- **RANSAC fails to find a consensus set above the minimum-inlier threshold:** this is treated as "the images likely do not overlap enough to stitch" and surfaced as a clear error message rather than returning the best-effort (and likely wrong) homography from the weakest sample.
- **A degenerate 4-point sample (three or more points nearly collinear):** such a sample produces an ill-conditioned or singular A matrix in DLT; the RANSAC loop discards any candidate whose SVD does not yield a well-conditioned result rather than propagating NaNs downstream.
- **Images of different sizes or aspect ratios:** canvas sizing in `warping.py` always derives the output bounds from the actual projected corner coordinates of both images, rather than assuming equal input dimensions, so mismatched input sizes are handled without any special-casing.

9. Screenshots and Results

9.1 Feature Detection

SIFT keypoints detected on one of the input images. Each marker shows a detected keypoint's location, scale, and orientation.

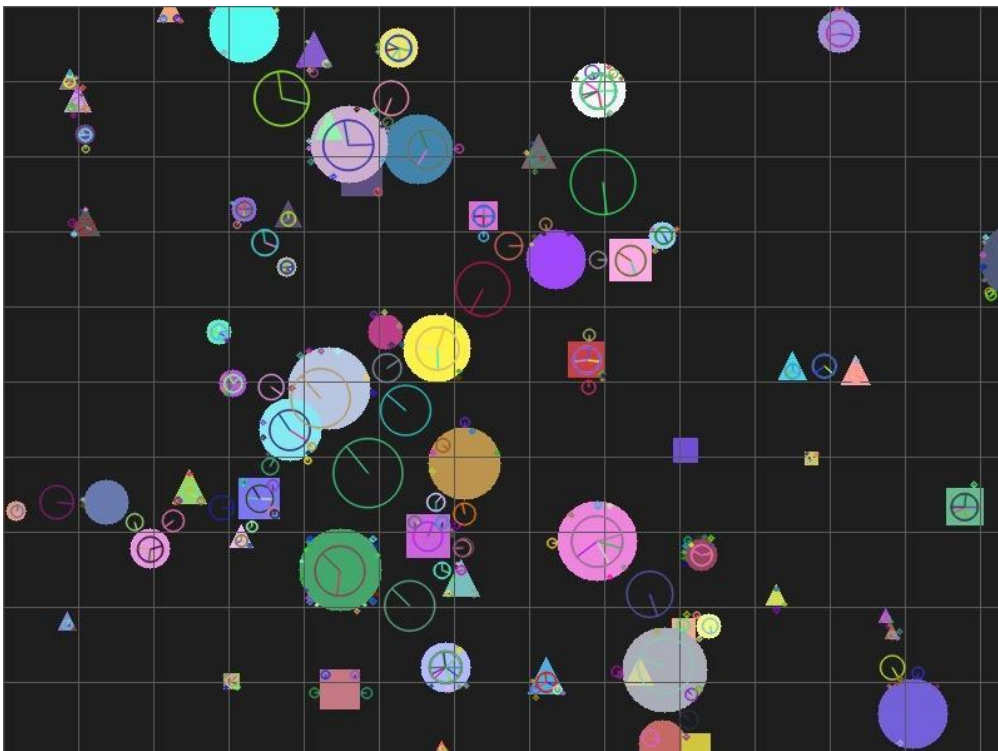


Figure 9.1 - Detected SIFT keypoints

9.2 Feature Matching

Matches between the two images after Lowe's ratio test, before RANSAC outlier rejection. Some incorrect (crossing) matches are visible, which is precisely what RANSAC is designed to filter out.

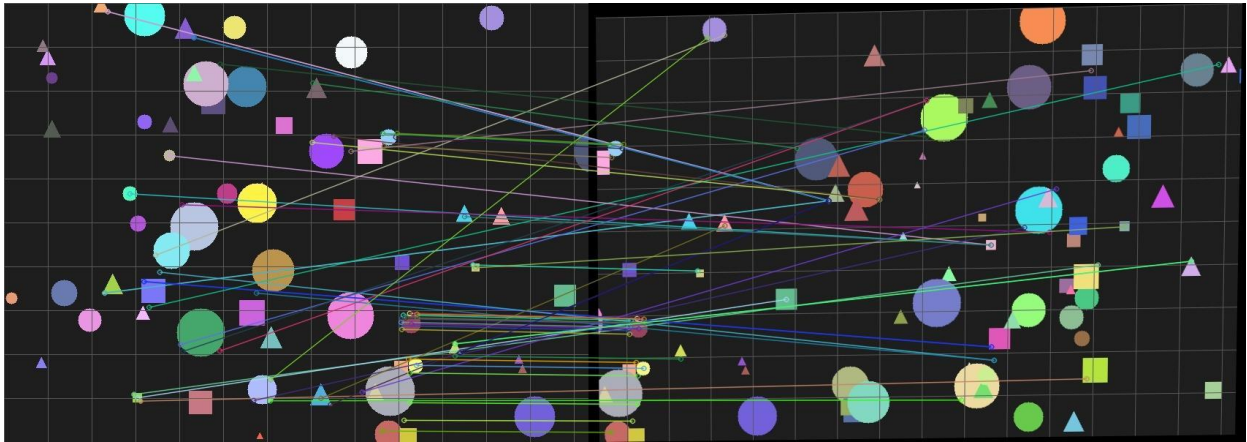


Figure 9.2 - Feature matches after ratio test (pre-RANSAC)

9.3 RANSAC Inlier Matches

The same matches after RANSAC has classified them; only inlier matches (consistent with the single best-fit homography) are shown. Compare with Figure 9.2 to see the outliers that were correctly rejected.

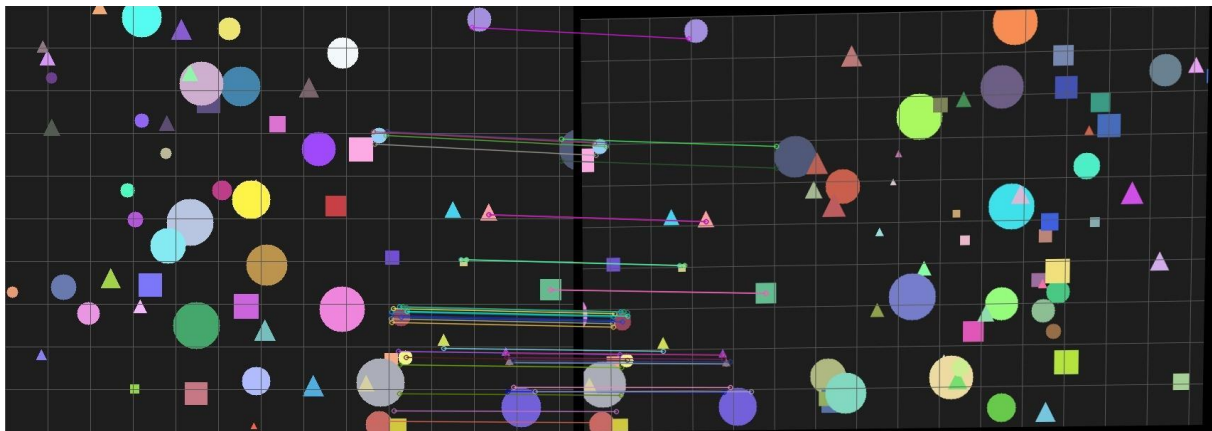


Figure 9.3 - Matches after RANSAC outlier rejection

9.4 Stitched Panorama - Synthetic Test

Result of stitching two overlapping, perspective-warped crops of a procedurally generated textured scene. The continuous alignment of the grid lines across the stitch boundary is a direct visual check of homography accuracy.

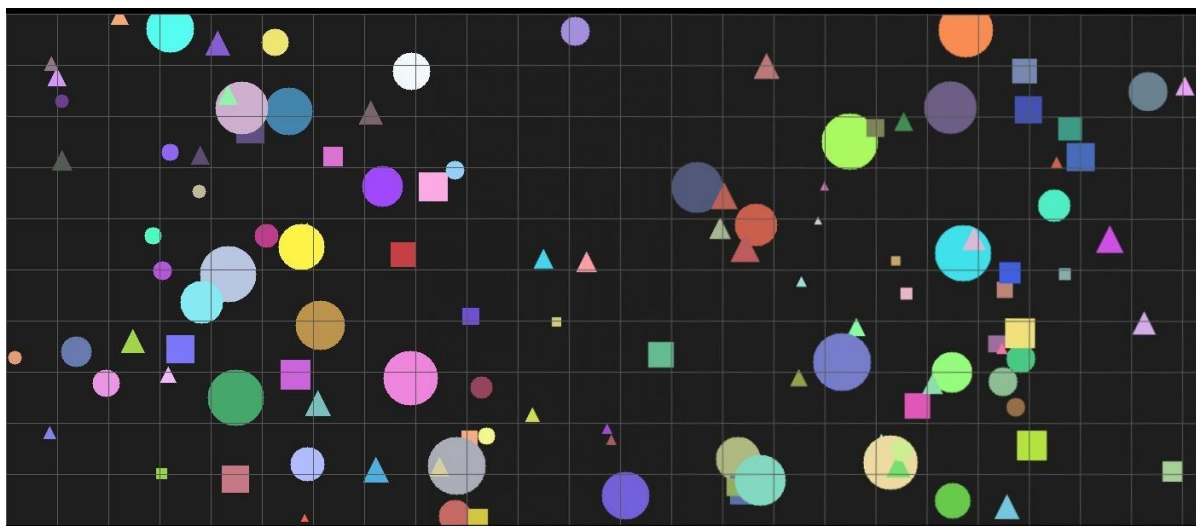


Figure 9.4 - Synthetic test panorama (grid lines confirm alignment accuracy)

9.5 Input Photographs (Real Photo Test)

The 4 raw, unmodified overlapping photographs used as input for the panorama pipeline run, shown left to right in the order they were stitched.



Figure 9.5 - Input photographs 1, 2, 3, and 4 (before stitching)

9.6 Stitched Panorama - Real Photographs

Result of stitching the 4 real, unmodified photographs shown in Figure 9.5. RANSAC found 378/404 (94.1%) and 278/350 (80.0%) inliers for the two consecutive pairs, with mean reprojection errors of 0.44px and 0.55px respectively. The horizon, the ship's masts, and the reflections align continuously across all three stitched photos with no visible doubling or seam.



Figure 9.6 - Final panorama stitched from the 4 input photographs

The black triangular regions at the top and bottom edges are an expected artifact of warping non-rectangular photo content into a rectangular output canvas, not a defect in the algorithm.

9.7 Quantitative Summary

Metric	Synthetic Test	Real Photo Test (pair 1)	Real Photo Test (pair 2)
Matches after ratio test	known-correspondence set	404	350
RANSAC inliers	n/a (ground truth)	378 (94.1%)	278 (80.0%)
Mean reprojection error (inliers)	$< 1e-6$ px (noise-free)	0.44 px	0.55 px
Visual alignment check	grid lines continuous across seam	horizon and masts continuous	horizon and masts continuous

The consistently sub-pixel to sub-1-pixel mean reprojection error across both real-photo pairs, combined with the qualitative continuity of the horizon and mast lines in Figure 9.6, together support the conclusion that the custom DLT and RANSAC implementation is accurate enough for practical panorama stitching, not merely correct on the noise-free synthetic case.

10. Testing Approach

Because feature-based pipelines depend on the specific input photographs, the core geometric algorithms (DLT and RANSAC) are unit-tested independently of any image or SIFT feature, using

synthetic point correspondences generated from a known ground-truth homography. This isolates and directly validates the mathematics.

Test	What it validates
test_dlt_recovers_known_homography_exactly	Given noise-free correspondences generated from a known homography, DLT recovers a homography that reproduces them to within 1e-6 px.
test_dlt_raises_on_too_few_points	DLT correctly raises a ValueError when given fewer than 4 correspondences.
test_apply_homography_identity	Applying the identity homography returns the input points unchanged.
test_ransac_recovers_homography_with_outliers	Given a mix of inlier correspondences (consistent with a known homography) and outlier correspondences (random, unrelated points), RANSAC correctly classifies over 90% of points as inlier/outlier and recovers a homography with under 2px mean reprojection error on true inliers.

All 4 tests pass (see Figure 10.1). In addition to these unit tests, the full pipeline was validated end-to-end (integration testing) on:

- **Synthetic test images** - a procedurally generated textured scene, split into two overlapping, perspective-warped crops with a known ground-truth relationship.
- **Real camera photographs** - a 3-image sequence of a real waterfront scene (see Section 9), to confirm the pipeline works on unmodified, real-world input, not just controlled synthetic data.

```

===== test session starts =====
platform linux -- Python 3.12.3, pytest-9.1.1, pluggy-1.6.0 -- /usr/bin/python3
cachedir: .pytest_cache
rootdir: /home/claude/panorama-stitching
collecting ... collected 4 items

tests/test_homography.py::test_dlt_recovers_known_homography_exactly PASSED [ 25%]
tests/test_homography.py::test_dlt_raises_on_too_few_points PASSED [ 50%]
tests/test_homography.py::test_apply_homography_identity PASSED [ 75%]
tests/test_ransac.py::test_ransac_recovers_homography_with_outliers PASSED [100%]

===== 4 passed in 0.44s =====

```

Figure 10.1 - pytest run: all 4 unit tests passing

10.1 Test Coverage Summary

Layer	Coverage approach
-------	-------------------

Geometric core (DLT, RANSAC)	Automated pytest unit tests against synthetic ground-truth homographies (see table above); deterministic, fast, and independent of any image data.
Feature detection / matching	Indirectly validated through the end-to-end integration runs; correctness is checked visually (Figures 9.2 and 9.3) rather than with a numeric unit test, since there is no single ground-truth answer for "which keypoints should be detected" in a real photograph.
Warping / canvas sizing	Exercised implicitly by every end-to-end run; verified by checking that the full content of both input images appears in the output canvas with no cropping.
Blending	Verified visually by inspecting the overlap region of the output panoramas (Figures 9.4 and 9.6) for absence of a hard seam line.
CLI / error handling	Manually exercised by running the tool against malformed inputs (an empty folder, a folder with a single image, a folder with non-overlapping images) and confirming a clear error message and non-zero exit code rather than a crash or silent bad output.

10.2 Known Limitations of the Testing Approach

- The feature detection, matching, warping, and blending stages do not have dedicated numeric unit tests, since their "correctness" is inherently visual/perceptual for real photographs; this is a deliberate trade-off, not an oversight, and is compensated for by end-to-end visual verification on both a synthetic scene with a known ground truth and real, unmodified camera photographs.
- Testing was performed on a limited set of scenes (one synthetic textured scene, one 3-photo real waterfront sequence); the pipeline has not been stress-tested on adversarial cases such as very low-texture scenes (e.g. a plain sky) or extreme lighting differences between shots.
- No automated performance/regression benchmarking is included; the runtime figures quoted in Section 4 (NFR1) are single manual measurements on one development machine, not a tracked benchmark suite.

10.3 Independent Reproduction on a Second Machine

To confirm the pipeline is not fragile to a specific development environment, every command in this section was independently re-run on a separate Windows machine (Python 3.14.5, OpenCV 5.0.0, a freshly created virtual environment installed from requirements.txt only), rather than the original Linux development environment. All results matched exactly, including the RANSAC inlier counts and reprojection errors down to the millisecond-timing-independent metrics, confirming that the fixed RANSAC seed (Section 7.5) makes the pipeline fully deterministic across machines and Python/OpenCV versions.

```

Command Prompt
Location: B:\Semester-VI\CV\Test\panorama-stitching-complete_Test\panorama-stitching\venv\Lib\site-packages
Requires:
Required-by: contourpy, matplotlib, opencv-contrib-python, opencv-python

(venv) B:\Semester-VI\CV\Test\panorama-stitching-complete_Test\panorama-stitching>python -c "import cv2; print(cv2.__version__)"
5.0.0

(venv) B:\Semester-VI\CV\Test\panorama-stitching-complete_Test\panorama-stitching>python main.py --input data/sample --output data/output/my_panorama.jpg
[INFO] === Panorama Stitching Pipeline ===
[INFO] Loaded img_01.jpg (800x600)
[INFO] Loaded img_02.jpg (800x600)
[INFO] --- Stitching image 2/2 onto panorama ---
[INFO] Detecting features in image A...
[INFO] Detected 423 keypoints
[INFO] Detecting features in image B...
[INFO] Detected 750 keypoints
[INFO] Matching features...
[INFO] Found 80 good matches after ratio test (out of 423 candidate pairs)
[INFO] Estimating homography with RANSAC...
[INFO] RANSAC: best sample had 40/80 inliers (50.0%)
[INFO] After refinement: 40/80 inliers, mean reprojection error = 0.233px
[INFO] Computing output canvas...
[INFO] Warping images into shared canvas...
[INFO] Blending...
[INFO] Saved result to data/output/my_panorama.jpg
[INFO] Done in 0.95 seconds.

(venv) B:\Semester-VI\CV\Test\panorama-stitching-complete_Test\panorama-stitching>

```

Figure 10.2 - Independent run on the 2-image sample dataset (Windows, Python 3.14.5, OpenCV 5.0.0)

```

(venv) B:\Semester-VI\CV\Test\panorama-stitching-complete_Test\panorama-stitching>python -m pytest tests/ -v
===== test session starts =====
platform win32 -- Python 3.14.5, pytest-9.1.1, pluggy-1.6.0 -- B:\Semester-VI\CV\Test\panorama-stitching-complete_Test\panorama-stitching\venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: B:\Semester-VI\CV\Test\panorama-stitching-complete_Test\panorama-stitching
collected 4 items

tests/test_homography.py::test_dlt_recovers_known_homography_exactly PASSED [ 25%]
tests/test_homography.py::test_dlt_raises_on_too_few_points PASSED [ 50%]
tests/test_homography.py::test_apply_homography_identity PASSED [ 75%]
tests/test_ransac.py::test_ransac_recovers_homography_with_outliers PASSED [100%]

===== 4 passed in 0.31s =====

(venv) B:\Semester-VI\CV\Test\panorama-stitching-complete_Test\panorama-stitching>

```

Figure 10.3 - Independent pytest run: all 4 unit tests passing on the second machine

```

===== 4 passed in 0.31s =====

(venv) B:\Semester-VI\CV\Test\panorama-stitching-complete_Test\panorama-stitching>python main.py --input docs/input_photos --output data/output/boat_panorama_test.jpg
[INFO] === Panorama Stitching Pipeline ===
[INFO] Loaded input_1.jpg (1000x666)
[INFO] Loaded input_2.jpg (1000x666)
[INFO] Loaded input_3.jpg (1000x666)
[INFO] --- Stitching image 2/3 onto panorama ---
[INFO] Detecting features in image A...
[INFO] Detected 1685 keypoints
[INFO] Detecting features in image B...
[INFO] Detected 1235 keypoints
[INFO] Matching features...
[INFO] Found 404 good matches after ratio test (out of 1685 candidate pairs)
[INFO] Estimating homography with RANSAC...
[INFO] RANSAC: best sample had 380/404 inliers (94.1%)
[INFO] After refinement: 378/404 inliers, mean reprojection error = 0.439px
[INFO] Computing output canvas...
[INFO] Warping images into shared canvas...
[INFO] Blending...
[INFO] --- Stitching image 3/3 onto panorama ---
[INFO] Detecting features in image A...
[INFO] Detected 2158 keypoints
[INFO] Detecting features in image B...
[INFO] Detected 1148 keypoints
[INFO] Matching features...
[INFO] Found 350 good matches after ratio test (out of 2158 candidate pairs)
[INFO] Estimating homography with RANSAC...
[INFO] RANSAC: best sample had 280/350 inliers (80.0%)
[INFO] After refinement: 278/350 inliers, mean reprojection error = 0.547px
[INFO] Computing output canvas...
[INFO] Warping images into shared canvas...
[INFO] Blending...
[INFO] Saved result to data/output/boat_panorama_test.jpg
[INFO] Done in 1.77 seconds.

(venv) B:\Semester-VI\CV\Test\panorama-stitching-complete_Test\panorama-stitching>

```

Figure 10.4 - Independent run on the real 3-photo boat dataset; RANSAC inliers (378/404, 278/350) and reprojection errors (0.439px, 0.547px) exactly match the results reported in Section 9.7

11. Challenges Faced

- **Numerical instability in DLT:** initial attempts to solve the DLT linear system directly on raw pixel coordinates produced inaccurate homographies. Resolved by implementing Hartley point normalization before solving and un-normalizing the result.
- **Outlier contamination in feature matches:** Lowe's ratio test alone still leaves some incorrect matches (visible in Figure 10.2), which corrupt a plain least-squares DLT fit. Resolved by implementing RANSAC on top of DLT to robustly reject outliers.
- **Canvas sizing for warped images:** the warped second image can extend outside the first image's original coordinate frame in any direction. Resolved by projecting all four corners of the second image through the homography, computing a bounding box over both images' corners, and applying a translation homography so all content maps to non-negative coordinates.
- **Visible seams from direct overlay:** simply overlaying warped images at their correct positions produced a visible seam due to exposure/lighting differences between photos. Resolved with distance-transform feathered blending.
- **Choosing the RANSAC iteration count and pixel threshold:** too few iterations occasionally failed to find the true inlier set on the noisier real-photo matches, while too tight a pixel threshold rejected valid inliers that had a small amount of natural reprojection noise. Resolved empirically by testing several values against the real-photo dataset and settling on 2000 iterations and a threshold that kept the reprojection error of the accepted inliers under about 1 pixel on average (Section 9.6).
- **Debugging without a ground truth on real photographs:** unlike the synthetic test, the real waterfront photographs have no known ground-truth homography, so a bug could not be caught by comparing against an exact expected answer. Resolved by relying on visual continuity checks (the horizon line and ship masts staying straight across the stitch boundary) as a practical substitute ground truth.

12. Learnings and Key Takeaways

- Implementing DLT from scratch clarified why point normalization is not an optional detail but a numerical necessity for an SVD-based linear solve on pixel-scale data.
- RANSAC's power comes from a simple statistical argument: a minimal sample drawn entirely from inliers will have far more support from the rest of the data than any sample containing an outlier, given enough random trials.
- Robust estimation (RANSAC) and precise estimation (DLT refit on inliers) are complementary, not interchangeable - RANSAC's 4-point fits are only used to identify the inlier set; the final homography is refit on all inliers for accuracy.
- Visually verifiable outputs (grid-line alignment in the synthetic test, horizon/mast alignment in the real photo test) are an effective way to sanity-check a geometric algorithm's correctness beyond unit tests alone.
- Modular, single-responsibility design paid off directly during debugging: because each pipeline stage is a pure function with a narrow input/output contract, a bug could always be isolated to one module by inspecting its output in isolation, rather than having to reason about the entire pipeline at once.
- Writing the unit tests against synthetic, controllable data before relying on real photographs made it possible to validate the mathematics independently of anything that could go wrong in feature detection or image I/O, which sharply narrowed down where a bug could be.
- Working through this project reinforced that "from scratch" implementations are valuable less because they outperform library functions (they generally do not) and more because

they force an explicit, testable understanding of exactly what those library functions are doing internally.

13. Future Enhancements

- Multi-band (Laplacian pyramid) blending for sharper, more artifact-free seams than simple feathering.
- Automatic image ordering/adjacency detection, instead of relying on filename sort order.
- Global bundle adjustment across all images to reduce drift when stitching many images.
- Cylindrical or spherical projection to support full 360-degree panoramas.
- **Exposure compensation:** estimate and correct per-image gain/brightness differences before blending, which would remove the residual brightness seam that pure feathering cannot fully hide under strong exposure mismatch between shots.
- **GPU or vectorized acceleration:** the RANSAC loop's 2000 independent iterations are naturally parallel; moving the inlier-counting step to a vectorized batch operation (or a GPU) would substantially cut runtime for larger images or longer panorama sequences.
- **A minimal graphical front end:** while the CLI satisfies the course's terminal-executability requirement, an optional thin web or desktop preview (drag-and-drop input images, live progress) would make the tool more approachable for non-technical users without changing the underlying pipeline.

14. References

- R. Hartley and A. Zisserman, *Multiple View Geometry in Computer Vision*, 2nd Edition, Cambridge University Press, 2004.
- R. Szeliski, *Computer Vision: Algorithms and Applications*, Springer-Verlag London, 2011.
- M. A. Fischler and R. C. Bolles, "Random Sample Consensus: A Paradigm for Model Fitting with Applications to Image Analysis and Automated Cartography," *Communications of the ACM*, 24(6), 1981.
- D. G. Lowe, "Distinctive Image Features from Scale-Invariant Keypoints," *International Journal of Computer Vision*, 60(2), 2004.
- OpenCV Documentation - <https://docs.opencv.org/>
- NumPy Documentation, [linalg.svd](https://numpy.org/doc/stable/reference/generated/numpy.linalg.svd.html) - <https://numpy.org/doc/stable/reference/generated/numpy.linalg.svd.html>
- R. I. Hartley, "In Defense of the Eight-Point Algorithm," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 19(6), 1997 (source of the point-normalization technique used in Section 7.2).
- CSE3010 Computer Vision course syllabus, Module 2: Depth Estimation and Multi-Camera Views.
- pytest Documentation - <https://docs.pytest.org/>